Audio Emotion Recognition (AER) Encoder

This repository contains the completed, fully-featured codebase for the Audio Emotion Recognition (AER) Encoder. This system uses acoustic deep learning to convert raw speech waves into categorized emotions (e.g., Happy, Sad), continuous coordinates (Valence and Arousal), and a dense Latent Vector representation designed explicitly to be consumed by downstream Vector Databases for conversational search matching.

Below is an extensive, plain-English breakdown of every file, its architecture, and exactly what it does under the hood.

---

1. System Configuration

`src/config/config.py`

Purpose: Acts as the central brain controlling all settings and folder paths for the entire application, eliminating "magic numbers" hardcoded in random files.

- **Model Parameters:** Defines that we are using `facebook/wav2vec2-base` from Hugging Face and utilizing a strictly **7-class categorical label layout** (0: Neutral, 1: Happy, 2: Sad, 3: Angry, 4: Fear, 5: Surprise, 6: Disgust).
- **Audio Constraints:** Defines our strict rule that all incoming audio is forced to be resampled to 16kHz and cut/padded to a maximum duration of 5 seconds.
- **Path Auto-Generation:** It dynamically links your `base_dir` so that paths like `data/raw/generic` always work, creating them automatically if they are missing.

---

2. Dataset Preprocessing & Standardization

`scripts/preprocess_datasets.py`

Purpose: Standardizes completely different external datasets (RAVDESS, TESS, CREMA, SAVEE) so they can all be trained together seamlessly.

- **What it does:**
  1. It recursively hunts through the `data/raw/generic` folder for any `.wav` file.
  2. Because each university labels their files differently (e.g., TESS uses folder names like `OAF_angry`, but RAVDESS uses a specific number code directly inside the filename like `03-01-05`), this script uses custom rules per dataset to figure out what emotion is present.
  3. It explicitly maps bizarre labels down to our unified `0-6` class array.
  4. It automatically assigns approximate Valence/Arousal values statically linked to the mapped label.
  5. It outputs a huge Excel-style spreadsheet called `train_manifest.csv` which lists every file and its clean label, along with a bar-chart picture `class_distribution.png` showing exactly how balanced the dataset currently is.

`src/data/dataset.py`

Purpose: The engine that feeds data securely into PyTorch during training.

- **What it does:**

1. **Audio Re-sampling:** When it loads a `.wav` file, it checks the sample rate. If a file is 48kHz, it mathematically down-samples it down to 16kHz to prevent the model from crashing.
2. **Mono-Conversion:** If an audio file comes in stereo (2 channels - Left and Right ear), it averages them into a Single Mono-channel (1 channel), because Wav2Vec2 only analyzes single-track sound.
3. **Padding/Truncating:** It acts like scissors. If an audio clip is 8 seconds long, it snips it down to 5 seconds. If it's only 2 seconds long, it pads the rest with complete silence (zeros) so all memory blocks are physically identical when handed to the GPU.

---

3. Core Artificial Intelligence (The Architecture)

`src/models/affective_encoder.py`

Purpose: The proprietary layer wrapped around the Hugging Face model that controls how the audio is evaluated.

- **What it does:**
  1. Loads `facebook/wav2vec2-base` but strips away the standard "Transcription / Speech-to-Text" capability.
  2. **Freezes Extractor:** It "freezes" the foundational Convolutional Neural Network (CNN) layers of Wav2Vec2. This means the model's base understanding of what sound physically is (like pitch and tone) is protected and won't get accidentally overwritten during training.
  3. **Multi-Task Heads:** It splits the architecture's final output into two distinct funnels:
     - The `categorical_classifier`: Determines which of the 7 emotion labels this is.
     - The `dimensional_classifier`: Determines continuous values for Valence and Arousal simultaneously.
  4. **The Latent Mapping: This is the most critical block.** Inside the `forward()` pass function, we command the model with `output_hidden_states=True`. This takes the massively complex neural

relationships formed right before the final prediction outputs and essentially mathematically squashes them down using "Mean Pooling". This squashed representation is outputted as a literal 768-number array (the `latent_vector`).

---

4. Execution Pipeline (Training & Evaluation)

`scripts/train.py`

Purpose: Trains the system and adjusts the weights so it actually learns the emotions.

- **What it does:**
  1. Sets up the device automatically. It checks if an NVIDIA GPU (`cuda`) is present, if an Apple CPU (`mps`) is present, or defaults to a basic processor (`cpu`).
  2. **Multi-Loss Strategy:** In the `train_epoch` backpropagation loop, it calculates error differently for different things. It uses `CrossEntropyLoss` to see how bad it failed predicting the emotion, and `MSELoss` (Mean Squared Error) to see how far off its Valence/Arousal estimates were. It combines both errors into one overarching `loss` score and uses that single score to penalize and correct the model weights step-by-step.

`scripts/evaluate.py`

Purpose: Tests the model cleanly outside the training loop to see how well it's truly performing on data it hasn't seen.

- **What it does:** Returns human-readable metric totals: pure accuracy percentages (for the 7 emotion labels) and Mean Absolute Error (MAE) averages to tell you how far off its Valence and Arousal scores are straying dynamically.

---

5. Live Production

`scripts/inference.py`

Purpose: The final endpoint mimicking live production. This is the script you would hit locally or via a web server to test a single user's audio snippet.

- **What it does:**
    1. Accepts a `.wav` file input from the command line.
    2. Feeds it through the prep-and-padding functions inside `dataset.py`.
    3. Secures a `.eval()` prediction output without triggering training math or memory leaks.
    4. Formats everything beautifully into a `JSON payload` that prints the String label (e.g., "Fear"), the floating-point Valence/Arousal values, and slices off the top of the `768-D Latent Vector` to prove that the massive number representation is ready to be stored into the Vector Database Search Engine down the line!